

Grundlagen Theoretischer Informatik

by Barbara Hammer

Summer semester 2026

Lecture notes by Liliana Sanfilippo

April 2026

Contents

1	Formale Sprachen und Grammatiken	5
1.1	Historisches	5
1.2	Kapitelinhalt	5
1.3	Wortersetzungssysteme und Notationen	6
1.4	Reguläre Sprachen	8
1.4.1	Lineare Grammatiken	9
1.4.2	Strukturelle Eigenschaften	9
1.5	Reguläre Sprachen	9
1.6	Kontextfreie Sprachen	9
2	Weiteres	11
	Backmatter	i
	Figures	i
	Index	iii
	Definitions	iii
	References	v

Formale Sprachen und Grammatiken

Formale Sprachen referieren im Gegensatz zu natürlichen Sprachen auf Sprachen in der Technik: wie unterhalte ich mich mit einem Rechner oder Roboter? Eins der wichtigsten Beispiele sind Programmiersprachen auf unterschiedlichem Niveau, angefangen von Assembler über gängige Hochsprachen wie Java oder C++ zu Anwendungssprachen wie etwa Matlab. Andere wichtige Bereiche, in denen formale Sprachen eine zentrale Rolle spielen, sind etwa XML, oder aber direkte Maschinenbefehle etwa in der NC Steuerung. Anders als natürliche Sprache muss dieses so präzise sein, dass die Maschine keinerlei Interpretationsspielraum hat und genau das versteht, was der Mensch intendiert. Zusätzliche Anforderung ist, dass dieses auch möglichst effizient umgesetzt werden kann, d.h. der Rechner sollte ein Wort möglichst schnell erkennen und interpretieren können.

0_orga.pdf

Vorerst irrelevant.

1.1 | Historisches

1_theinf_motivation.pdf - Skript S. 1–2

Die Formalisierung von Sprache ist ein nicht nur für die Informatik im Rahmen technischer Systeme relevantes Gebiet, sondern auch im Rahmen der Sprachphilosophie und Linguistik behandelt worden. So ist einer der wichtigsten Forscher in dem Gebiet Noam Chomsky, ein Linguist, der 1956 die nach ihm benannte Chomsky-Hierarchie von formalen Sprachen einführte (Chomsky 1956). Diese unterscheidet unterschiedlich komplexe Formalisierungen, die zu unterschiedlich komplexen Sprachkonstrukten führen. Diese theoretischen Überlegungen hatten in der Informatik direkte Bedeutung für die Entwicklung von Programmiersprachen. Die erste Programmiersprache, für die eine formale Syntax eingeführt wurde, war um 1960 ALGOL 60. Unter anderem war hier John Backus beteiligt, nach dem die heute noch gebräuchliche Backus-Naur Form von Grammatiken benannt ist. Basierend auf dieser Formalisierung war die Entwicklung von Compilern deutlich einfacher und eleganter möglich.

Das Gebiet der Formalen Sprachen behandelt unter anderem auch, was auf unterschiedlichen Komplexitätsstufen darstellbar ist und welche strukturellen Eigenschaften gelten. Interessanterweise sind einige wichtige strukturelle Eigenschaften in dem Gebiet auch noch offen bzw. teilweise sehr spät gelöst worden. Der Abschluss so genannter kontextsensitiver Sprachen gegen Komplementbildung wurde erst 1988 bewiesen im Satz von Immerman/Szelepcsényi (Immerman 1988). Letzterer war damals noch Student. Er erhielt 1995 zusammen mit Immerman hierfür den Gödel-Award, den Nobelpreis der Informatik.

1.2 | Kapitelinhalt

2_0_sprachen_intro.pdf - Skript S. 2

In diesem Kapitel werden wir eine Hierarchie von formalen Sprachen auf der Basis von Grammatiken und deren Komplexität kennenlernen. Das dient dazu, unterschiedlich komplexe formale Sprachen definieren und interessante Eigenschaften wie etwa den Abschluss untersuchen zu können.

1.3 | Wortersetzungssysteme und Notationen

2_1_Sprachen_Grundlagen.pdf - Skript S. 2–6

Wortersetzungssysteme oder auch Semi-Thue-Systeme bilden ein grundlegendes Konzept, Wörter zu erzeugen oder abzuändern. Ausgehend von einem Startsymbol kann man eine Menge komplexer Wörter durch entsprechende Ersetzungsregeln definieren. Wir werden Wortersetzungssysteme als Grundlage in der gesamten Vorlesung verwenden. Zunächst geben wir einige Definitionen:

Definition 1.1 Symbol

todo

Definition 1.2 Alphabet

Endliche Menge von Symbolen:

$$\Sigma = \{a_1, \dots, a_n\}$$

Dabei ist Σ^* die Menge aller Wörter über Σ und Σ^m die Menge aller Wörter der Länge m über Σ .

Definition 1.3 Wort

Eine endliche Folge von Elementen aus einem Alphabet:

$$\bar{w} = a_{i_1} \dots a_{i_m}, \quad a_{i_j} \in \Sigma, \quad m > 0$$

Dabei ist $m = \bar{w}$ die Länge des Wortes und ein Wort der Länge 0 ist das leere Wort ϵ

Definition 1.4 Anzahl der Vorkommnisse des Symbols in einem Wort

Die Anzahl der Vorkommnisse des Symbols a_i im Wort $\bar{w}$.

Definition 1.5 leeres Wort

Wort der Länge 0, notiert durch ϵ

Definition 1.6 Σ^*

Menge aller Wörter über Σ

Definition 1.7 Σ^m

Menge der Wörter der Länge m über Σ

Definition 1.8 Konkatenation

Eine Verkettung zweier Wörter $\bar{w} = a_{i_1} \dots a_{i_m}$ und $\bar{v} = a_{i_1} \dots a_{i_l}$ zu

$$\bar{w}\bar{v} = a_{i_1} \dots a_{i_m} a_{i_1} \dots a_{i_l}$$

Definition 1.9 Präfix $\bar{w}$ von $\bar{u}$

Wenn $\bar{u} = \bar{w}\bar{v}$

Definition 1.10 Postfix $\bar{v}$ von $\bar{u}$

Wenn $\bar{u} = \bar{wv}$

Definition 1.11 Monoid

Assoziative Strukturen, die ein neutrales Element haben und für die eine assoziative Operation mit Einselement existiert

Satz 1.1 Aus der Gleichheit $\bar{u}\bar{v} = \bar{u}\bar{w}$ oder $\bar{v}\bar{u} = \bar{w}\bar{u}$ folgt $\bar{v} = \bar{w}$

Notation 1.1 Potenzen von Wörtern $\bar{w}$ schreiben wir kurz als $\bar{w}^k = \underbrace{\bar{w} \dots \bar{w}}_{k \text{ mal}}$.

Beispiel 1.1 Zu Definition 1.2.

$\{0, 1\}$, $\{a, b, c\}$, $\{4711, ha, ende\}$ sind **Alphabets**, wobei jeder Eintrag als ein untrennbares Zeichen gewertet wird.

$\{0, 1, 2, 3, \dots\}$, ist kein gültiges **Alphabet**, weil die Menge unendlich groß ist.

$\{a, aa, aaa\}$ ist zwar ein gültiges **Alphabet** nach unserer Definition, aber nicht empfehlenswert, weil es keine eindeutige Identifikation der Zeichen erlaubt.

Beispiel 1.2 Zu Definition 1.7 .

Alle Wörter über dem Alphabet $\{0, 1\}$ der Länge maximal 3, aufgeschrieben in lexikographischer Reihenfolge, sind

$\epsilon, 0, 1, 00, 01, 10, 11, 000, 001, 010, 100, 011, 110, 101, 111$

Notation 1.2 Um besser zwischen Variablen und Konstanten zu unterscheiden, schreiben wir konstante Wörter und Symbole unterstrichen und aufrecht (etwa **bla**). Variablen schreiben wir stattdessen kursiv. Außerdem heben wir Wortvariablen, ähnlich zu Vektoren in der linearen Algebra, durch einen Überstrich hervor (etwa $\bar{w}$).

Definition 1.12 Wortersetzungssystem über einem **Alphabet** Σ

Eine endliche Menge R von Ersetzungsregeln über Σ type

Bemerkung 1.1 Für den Begriff “Ersetzungsregel” gibt es reichlich Synonyme, z.B. “Regel”, “Grammatikregel”, “Substitution”, “Produktionsregel” und “Produktion”. Dementsprechend wird ein Wortersetzungssystem auch häufig “Regelmenge” oder “Produktionen” genannt

Definition 1.13 Ersetzungsregel über einem **Alphabet** Σ

Ein Tupel von Worten $\bar{\alpha}, \bar{\beta} \in \Sigma^*$, geschrieben als $\bar{\alpha} := \bar{\beta}$

Definition 1.14 Wort $\bar{v} \in \Sigma^*$ in einem Schritt ableitbar aus Wort $\bar{u} \in \Sigma^*$

Falls es eine Ersetzungsregel über einem Alphabet Σ $\bar{\alpha} ::= \bar{\beta}$ in R gibt mit $\bar{u} = \bar{u}_1 \bar{\alpha} \bar{u}_2$ und $\bar{v} ::= \bar{u}_1 \bar{\beta} \bar{u}_2$. Das heißt, $\bar{v}$ erhält man aus $\bar{u}$, indem man das gemäß einer Ersetzungsregel das Teilwort $\bar{\alpha}$ mit dem Teilwort $\bar{\beta}$ ersetzt.

Schreibweise: $\bar{u} \rightarrow_R^1 \bar{v}$ oder $\bar{u} \rightarrow_R \bar{v}$

Bemerkung 1.2 Falls das Wortersetzungssystem für die Ableitung aus dem Kontext klar ist, schreiben wir $\rightarrow_R$ auch als $\rightarrow$

Diese Relation gibt Anlass zu ihrer transitiven Hülle der in $t \in \mathbb{N}$ Schritten auseinander ableitbaren Wörter. Wir definieren:

Definition 1.15 in $t \in \mathbb{N}$ Schritten auseinander ableitbare Wörter

todo

Die transitive Hülle ist dann gegeben als die Menge

$$\{(\bar{u}, \bar{v}) \mid \bar{u}\}$$

Notation 1.3 Falls für irgendein

Definition 1.16 aus $\bar{u}$ ableitbare Wörter

$\bar{u} \rightarrow_R^0 \bar{v} \Leftrightarrow \bar{u} = \bar{v}$ und $\bar{u} \rightarrow_R^{t+1} \bar{v} \Leftrightarrow \exists \bar{u}' \in , \text{ mit } \bar{u} \rightarrow_R^1 \bar{u}', \bar{u}' \rightarrow_R^t \bar{v}$

Definition 1.17 formale Sprache

todo

Definition 1.18 Grammatik

todo

Definition 1.19 erzeugte Sprache

todo

Definition 1.20 Äquivalenz

todo

Definition 1.21 formale Regelsprache

todo

1.4 | Reguläre Sprachen

Definition 1.22 Verkettung

todo

Definition 1.23 Kleene-Stern

todo

Definition 1.24 Potenz

todo

Definition 1.25 regulärer Ausdruck

todo

Konvention 1.1 Wie in anderen Programmiersprachen üblich vereinbaren wir noch ein paar Kurzformen. Sei A ein regulärer Ausdruck über dem Alphabet $\Sigma = \{a_1, \dots, a_n\}$, dann ist:

Definition 1.26 reguläre Sprache

todo

reguläre Ausdrücke werden bei der Textverarbeitung häufig benutzt. Es ist daher interessant, möglichst einfache reguläre Ausdrücke für eine gegebene Sprache zu konstruieren. Ein Maß, das hier betrachtet wird, ist die so genannte Sternhöhe, also die Anzahl ineinander verschachtelter Operatoren *. Das Sternhöhenproblem war tatsächlich bis 1988 noch ungelöst; das ist die Frage, für eine gegebene Sprache einen regulären Ausdrücken zu finden mit möglichst wenig ineinander verschachtelten Operatoren *.

1.4.1 | Lineare Grammatiken

Definition 1.27 lineare Grammatik
todo

Definition 1.28 rechtslineare Grammatik
todo

Definition 1.29 linkslineare Grammatik
todo

Kommen wir zu einem “wüsten Beweis”, der **reguläre Ausdrücke** zu **linkslinearen Grammatiken** in Beziehung setzt (ein äquivalenter Beweis würde es erlauben, **rechtslineare Grammatiken** zu **regulären Ausdrücken** in Beziehung zu setzen, die Begriffe sind tatsächlich äquivalent):

Satz 1.2 Eine Sprache L über Σ ist regulär dann und nur dann, wenn es eine **rechtslineare Grammatik** G gibt mit $L(G) = L$.

Beweis Skript Seite 9–11.

1.4.2 | Strukturelle Eigenschaften

Für **reguläre Sprachen** gelten wichtige strukturelle Eigenschaften:

Satz 1.3 **Satz über die strukturellen Eigenschaften con regulären Sprachen**
reguläre Sprachen sind gegen Vereinigung, Schnitt, Konkatenation, Komplement und Kleene-Stern abgeschlossen, d.h. sind L und L' regulär, dann auch $L \cup L'$, $L \cap L'$, $L \circ L'$, $\Sigma^* \setminus L$ und L^* .

Beweis. Skript Seite 13.

Satz 1.4 Pumping-Lemma
todo

Beweis. Skript Seite 13–14.

1.5 | Reguläre Sprachen

2_2_Sprachen_Regulaer_Part1.pdf - Skript S. 6–13

2_2_Sprachen_Regulaer_Part2.pdf bis F. 57 - Skript S. 13–15

1.6 | Kontextfreie Sprachen

2_2_extra_pumpinglemma_beispiele.pdf - Extra

2_2_Sprachen_Regulaer_Part2.pdf ab F. 58 - Skript S. 15–19

Satz 1.5 Pumping-Lemma
todo

Beweis. Skript Seite 13–14.

Example 1.1 Zu Folie 7–8 aus 2_2_extra_pumpinglemma_beispiele.pdf

acbc abba ba

eine Zerlegung ist:

$$\underbrace{acbc}_{u} \underbrace{abba}_{v} \underbrace{ba}_{w}$$

eine andere wäre:

$$\underbrace{ac}_{u} \underbrace{bc}_{v} \underbrace{abbaba}_{w}$$

Das schafft man mit dem pumpen nur wenn in einem regex irgendwo ein Kleen Stern ist, alles vor dem stern ist das u , das wo der stern drauf angewendet wird ist v und das danach w . Wenn man sterne an mehrere Stellen setzen könnte, gibt es mehrere Zerlegungen.

Example 1.2 Zu F 58 an Tafel

Gegeben:

$$L_1 := \{a^n b^n \mid n \geq 0\}$$

$$L_2 := \{x \in \{a, b\}^* \mid \#a = \#b\}$$

$$L_3 := \{a^{n^2} \mid n \geq 0\}$$

Jemand behauptet: L_2 ist nicht regulär, denn $L_1 \subseteq L_2$ und L_1 ist nicht regulär.

Stimmt das? Nein, falsch!

Klingt intuitiv, aber Teilmengenbeziehungen einer **regulären Sprache** sagt nichts über die Regularität der anderen beteiligten Sprache aus.

Stattdessen macht man das mit einem formalen Widerspruchsbeweis (F 60):

Pumping-Lemma beschreibt wie reg sprachen aussehen, wenn die unendlich groß werden

Wichtigster Effekt/Ziel der Vorlesung: Sie will uns Formalierungen schmackhaft machen

CHAPTER 2

Weiteres

Backmatter

List of Figures

Definitions

Σ^m Menge der Wörter der Länge m über Σ . 6

Σ^* Menge aller Wörter über Σ . 6, 8

Anzahl der Vorkommnisse des Symbols in einem Wort Die Anzahl der Vorkommnisse des Symbols a_i im Wort $\bar{w}$. 6

erzeugte Sprache todo. 8

formale Regelsprache todo. 8

formale Sprache todo. 8

Grammatik todo. 8

Konkatenation Eine Verkettung zweier Wörter $\bar{w} = a_{i_1} \dots a_{i_m}$ und $\bar{v} = a_{i_1} \dots a_{i_l}$ zu

$$\overline{wv} = a_{i_1} \dots a_{i_m} a_{i_1} \dots a_{i_l}$$

. 6

leeres Wort Wort der Länge 0, notiert durch ϵ . 6, 7

lineare Grammatik todo. 9

Monoid Assoziative Strukturen, die ein neutrales Element haben und für die eine assoziative Operation mit Einselement existiert. 7

Postfix $\bar{v}$ von $\bar{u}$ Wenn $\bar{u} = \bar{wv}$. 7

Potenz todo. 8

Präfix Wenn $\bar{u} = \bar{wv}$. 6

Symbol todo. 6

Verkettung todo. 8

Wort Eine endliche Folge von Elementen aus einem Alphabet:

$$\bar{w} = a_{i_1} \dots a_{i_m}, \quad a_{i_j} \in \Sigma, \quad m > 0$$

Dabei ist $m = \bar{w}$ die Länge des Wortes und ein Wort der Länge 0 ist das leere Wort ϵ . 6

Äquivalenz todo. 8